Algorithmes et Pensée Computationnelle

Fonctions, méthodes, mémoire et exceptions

Le but de cette séance est d'approfondir vos connaissances en programmation. Au terme de cette séance, vous serez capable de :

- définir une fonction en Python ou une méthode en Java et l'utiliser dans un programme,
- utiliser des bibliothèques contenant des fonctions prédéfinies en Python ou des méthodes prédéfinies en Java,
- connaître la portée d'une variable,
- gérer des exceptions.

Pour les exercices où vous devez déterminer ce qu'affiche un programme, résolvez-les d'abord à la main avant d'utiliser IntelliJ, afin de simuler les conditions d'examen.

1 Opérations arithmétiques

Question 1: (*3 minutes*) [Java et Python] **Calculs (multiplication)**

Créez deux variables nommées `facteur1` et `facteur2` en Java (`facteur_1` et `facteur_2` en Python), avec pour valeurs respectives 11 et 3. Multipliez la première variable par la deuxième et stockez le résultat dans une nouvelle variable `produit`. Vous pouvez afficher les différentes variables pour voir leurs valeurs. Vous pouvez répéter l'exercice avec l'addition et la soustraction.

Conseil

L'opérateur de multiplication est le *, celui de l'addition est le + et celui de soustraction est le -.

Solution

Python :

```python
if __name__ == "__main__":
    facteur_1 = 11
    facteur_2 = 3
    produit = facteur_1*facteur_2
    print(facteur_1)
    print(facteur_2)
    print(produit)
```

Java :

```java
public class Question1 {
    public static void main(String[] args) {
        int facteur1 = 11;
        int facteur2 = 3;
        int produit = facteur1*facteur2;
        System.out.println(facteur1);
        System.out.println(facteur2);
        System.out.println(produit);
    }
}
```

Question 2: (*10 minutes*) [Java et Python] **Calculs (division)**
Créez deux variables nommées `nbBonbons` et `nbPersonnes` en Java (`nb_bonbons` et `nb_personnes`

en Python), avec pour valeurs respectives 11 et 3. Divisez la première variable par la deuxième et stockez le résultat dans une nouvelle variable nommée `bonbonsPersonnes` en Java (`bonbons_personnes` en Python). Pour finir, calculez le nombre de bonbons restants via l'opérateur % (modulo) et stockez le résultat dans une nouvelle variable `reste`. Vous pouvez afficher les différentes variables pour voir leurs valeurs. Chaque personne doit recevoir un nombre entier de bonbons.

Conseil

- Attention : en Python il existe deux opérateurs de division, / effectue une division classique, tandis que // effectue une division avec arrondi inférieur.
- En Java, lorsque les deux opérandes sont de type int, l'opérateur / effectue une division entière. Si au moins l'un des deux opérandes est de type float ou double, il effectue une division en virgule flottante.
- L'opérateur % (modulo) permet de calculer le reste d'une division. Par exemple, 10%2=0.

Solution

Python :

```python
if __name__ == "__main__":
    nb_bonbons = 11
    nb_personnes = 3
    bonbons_personnes = nb_bonbons // nb_personnes
    reste = nb_bonbons % nb_personnes
    print (nb_bonbons)
    print(nb_personnes)
    print(bonbons_personnes)
    print(reste)
```

Java :

```java
public class Question2 {
    public static void main(String[] args) {
        int nbBonbons = 11;
        int nbPersonnes = 3;
        int bonbonsPersonnes = nbBonbons / nbPersonnes;
        int reste = nbBonbons % nbPersonnes;
        System.out.println(nbBonbons);
        System.out.println(nbPersonnes);
        System.out.println(bonbonsPersonnes);
        System.out.println(reste);
    }
}
```

Question 3: (*5 minutes*) [Java et Python] **Calculs (incrémentation / décrémentation)**
Gardez vos variables de l'exercice précédent, augmentez de 1 la valeur de `nbBonbons` en Java (`nb_bonbons` en Python) et diminuez de 1 celle de `nbPersonnes` en Java (`nb_personnes` en Python).

Conseil

Vous pouvez utiliser les opérateurs += et -= en Python, et les opérateurs ++ (incrémentation) et −− (décrémentation) en Java.

Solution

Python :

```python
nb_bonbons = 11
nb_personnes = 3
nb_bonbons += 1
nb_personnes -= 1
bonbons_personnes = nb_bonbons // nb_personnes
reste = nb_bonbons % nb_personnes
print (nb_bonbons)
print(nb_personnes)
print(bonbons_personnes)
print(reste)
```

Java :

```java
public class Question3 {
    public static void main(String[] args) {

        int nbBonbons = 11;
        int nbPersonnes = 3;
        nbBonbons++;
        nbPersonnes--;
        int bonbonsPersonnes = nbBonbons / nbPersonnes;
        int reste = nbBonbons % nbPersonnes;
        System.out.println(nbBonbons);
        System.out.println(nbPersonnes);
        System.out.println(bonbonsPersonnes);
        System.out.println(reste);
    }
}
```

2 Variables, fonctions et méthodes

Question 4: (*5 minutes*) [Java **et** Python] **Fonctions et méthodes basiques**
Définissez une fonction en Python et une méthode en Java nommées `ping()` qui, lorsqu'elles sont appelées, affichent `pong`.

Conseil

Référez-vous aux diapositives dans ce dossier Moodle pour la création des fonctions en Python et des méthodes en Java.

Solution

Python :

```python
def ping() :
    print("pong")
if __name__ == "__main__":
    ping()
```

Solution

Java :

```java
public class Question4 {
    static void ping() {
        System.out.println("pong");
    }

    public static void main(String[] args) {
        ping();
    }
}
```

Question 5: (*5 minutes*) [Java et Python] **Fonction et méthode de multiplication**
Définissez une fonction en Python et une méthode en Java nommées `multiplicateur()` qui prennent deux arguments *multiple1* et *multiple2*, les multiplient et retournent le résultat. Stockez le résultat de `multiplicateur(2,3)` dans une variable *resultat* et affichez-la.

Conseil

— Référez-vous aux diapositives du cours pour la création et l'appel des fonctions en Python et des méthodes en Java.

— Pour retourner une valeur au lieu de l'afficher, utilisez le mot-clé `return` (pour Python et Java).

Solution

Python :

```python
def multiplicateur(multiple1, multiple2):
    return multiple1 * multiple2


if __name__ == "__main__":
    resultat = multiplicateur(2, 3)
    print(resultat)
```

Java :

```java
public class Question5 {
    public static int multiplicateur(int multiple1, int
    multiple2){
        return multiple1*multiple2;
    }

    public static void main(String[] args) {
        int resultat = multiplicateur(2, 3);
        System.out.println(resultat);
    }
}
```

Question 6: (*5 minutes*) [Java et Python] **Fonctions et méthodes pour l'aire et le périmètre**
Définissez deux fonctions en Python et deux méthodes en Java nommées `aire()` et `perimetre()`. Elles

prennent un argument (`rayon`) et renvoient respectivement l'aire et le périmètre d'un cercle. Stockez les résultats dans des variables `aire_resultat` et `perimetre_resultat` et affichez le contenu de ces variables.

Conseil

- Référez-vous aux diapositives du cours pour la création et l'appel des fonctions en Python et des méthodes en Java.
- Pour retourner une valeur au lieu de l'imprimer, utilisez le mot-clé `return` (pour Python et Java).
- Pour calculer a^b en Python, utilisez `a**b`, et `Math.pow(a, b)` en Java.
- Pour rappel, le périmètre d'un cercle s'obtient en utilisant la formule $P = 2 * \pi * r$ et l'aire s'obtient en utilisant la formule $A = r^2 * \pi$.

Solution

Python :

```
1  import math
2
3  def aire(rayon):
4    return (rayon**2)*math.pi
5
6  def perimetre(rayon):
7    return 2*math.pi*rayon
8
9  if __name__ == '__main__':
10     rayon = 10
11     aire_resultat = aire(rayon)
12     perimetre_resultat = perimetre(rayon)
13     print(f"L'aire d'un cercle de rayon {rayon} est égale à
       {aire_resultat}")
14     print(f"Le périmètre d'un cercle de rayon {rayon} est égal à
        {perimetre_resultat}")
```

Solution

Java :

```java
public class Question6 {

    static double aire(int rayon) {
        return Math.pow(rayon, 2) * Math.PI;
    }

    static double perimetre(int rayon) {
        return 2 * Math.PI * rayon;
    }

    public static void main(String[] args) {
        int rayon = 10;
        double aire = aire(rayon);
        double perimetre = perimetre(rayon);
        System.out.println("L'aire d'un cercle de rayon " +
    rayon + " est égale à " + aire);
        System.out.println("Le périmètre d'un cercle de rayon "
    + rayon + " est égal à " + perimetre);
     }
}
```

Question 7: (*10 minutes*) [Java] **Portée des variables**
Qu'affiche le programme suivant ?

```java
public class Question7 {
    static int a = 0;
    public static void f() {
        a += 2;
        System.out.println("a = " + a);
    }

    public static void g() {
        int b = 3;
        System.out.println("a = " + a);
        System.out.println("b = " + b);
    }

    public static void main(String[] args) {
        f();
        g();
        //System.out.println("b =" + b);
    }
}
```

Que se passerait-il si on décommente la ligne 17 (c'est-à-dire si l'on enlève les //) ?

Conseil

Tenir compte de l'endroit où chaque variable est définie ainsi que des opérations réalisées sur chaque variable.

Solution

— `a = 2`
`a = 2`
`b = 3`

— On obtiendra une erreur de compilation du programme car la variable `b` est créée à l'intérieur de la méthode `g()` et n'existe donc qu'au sein de celle-ci. La variable `b` n'étant pas définie en dehors de la méthode `g()`, celle-ci ne peut pas faire l'objet d'un appel en dehors de la méthode `g()`.

Question 8: (*5 minutes*) [Python] **Portée des variables**
Qu'affiche le programme suivant ?

```
1  x = 2
2
3  def fonction():
4      x = 3
5
6      def fonction2():
7          global x
8          print("x=" + str(x))
9
10     fonction2()
11     print("x=" + str(x))
12
13 print(fonction())
```

Conseil

— Le mot-clé `global` permet d'accéder aux variables globales (définies à l'extérieur de la fonction).

— Soyez attentif au type des éléments retournés par chacune des fonctions.

— En Python, une fonction qui atteint la fin de son exécution sans rencontrer d'instruction `return`, ou qui exécute une instruction `return` sans expression, renvoie implicitement `None`.

Solution

L'instruction `x = 2`, située en dehors des fonctions, définit une variable globale. L'instruction `x = 3`, dans `fonction()`, définit une variable locale distincte. Dans `fonction2()`, la déclaration `global x` indique que `x` désigne la variable globale : le premier appel à `print` affiche donc `x=2`. Cette déclaration ne s'applique qu'à `fonction2()`. Dans `fonction()`, `x` désigne toujours la variable locale : le deuxième appel à `print` affiche donc `x=3`. Enfin, `fonction()` ne contient aucune instruction `return` et renvoie donc implicitement `None`. L'appel `print(fonction())` affiche cette valeur après les deux lignes précédentes.

Le programme affiche :
```
x=2
x=3
None
```

3 Gestion des exceptions

Question 9: (*5 minutes*) [Python] **Types d'erreurs** Qu'affiche le programme suivant ?

```
value1 = "Algorithms"
value2 = 4

try:
    size = len(value1)
    result = size/value2
    print(f"Le résultat de la division est: {result}")

except Exception as error:
    print("On ne peut pas effectuer l'opération")

try:
    result = value1/2
    print(f"Le résultat de la division est: {result}")

except TypeError as error:
    print("On ne peut pas diviser une chaîne de caractères")
```

Conseil

Utilisée sur une chaîne de caractères, la fonction `len()` renvoie le nombre de caractères de la chaîne.

Solution

```
Le résultat de la division est: 2.5
On ne peut pas diviser une chaîne de caractères.
```

Question 10: (*5 minutes*) [Python] **Types d'erreurs** Qu'affiche le programme suivant ?

```
value1 = 4
value2 = ""

try:
    count = len(value2)
    result= value1/count
except ZeroDivisionError as error:
    print("Nous ne pouvons pas diviser un nombre par 0")
```

Conseil

La chaîne de caractères vide est représentée par ""

Solution

Nous ne pouvons pas diviser un nombre par 0.

Question 11: (*5 minutes*) [Python] **Types d'erreurs** Qu'affiche le programme suivant ?

```python
value1 = "Algorithms"
value2 = 4


try:
    decimal = float(value2)
except ValueError as error:
    print("Nous ne pouvons pas convertir un entier en décimal")

finally:
    try:
        value2 = int(value1)
    except ValueError as error:
        print("Nous ne pouvons pas convertir une chaîne de caractères en nombre")
```

Conseil

- `TypeError` survient lorsqu'une opération est effectuée sur des types incompatibles (exemple : `"abc" + 5`), tandis que `ValueError` survient lorsque le type est approprié, mais que la valeur est invalide (exemple : `int("abc")`).
- La fonction `float()` permet de convertir une variable en nombre à virgule flottante.
- La fonction `int()` permet de convertir une variable en nombre entier.

Solution

`float(value2)` est valide car on transforme un nombre entier en un float. Aucune erreur ne va être lancée.
`int(value1)` n'est pas valide si `value1` contient une chaîne qui ne représente pas un entier, comme `"abc"`. En revanche, une chaîne comme `"42"` peut être convertie en entier.

La chaîne `"Algorithms"` provoque ainsi une erreur avec le message suivant :
Nous ne pouvons pas convertir une chaîne de caractères en nombre

Question 12: (*5 minutes*) [Java] **Gestion d'erreurs** Le programme suivant est incorrect. Que devez-vous modifier pour qu'il fonctionne correctement ?

Conseil

Référez-vous à la diapositive 24 du cours de cette semaine.

```java
public class Question12 {

    public static void division(int a, int b) {
        if (b == 0) {
            throw new ArithmeticException();
        } else {
            float result = a / b;
            System.out.println("Le résultat de la division de " + a + "/" + b + "= " + result);
        }
    }

    public static void main(String args[]) {
        int value1 = 2;
```

```
14          int value2 = 4;
15          try {
16              division(value1, value2);
17              value2 = 0;
18              division(value1, value2);
19          } catch (IndexOutOfBoundsException err) {
20              System.out.println("Nous ne pouvons pas effectuer une
    division par 0");
21          }
22      }
23  }
```

Solution

À l'intérieur de la méthode `division`, lorsque `b` est égal à 0, le programme lève une exception de type `ArithmeticException`. Mais dans la méthode `main`, le bloc catch est configuré pour intercepter l'exception `IndexOutOfBoundsException`. Pour corriger cette erreur, une des solutions est de remplacer `IndexOutOfBoundsException` par `ArithmeticException`.

Question 13: (*5 minutes*) [Java] **Gestion d'erreurs**

1. Qu'affiche le programme suivant, en supposant que l'utilisateur saisit les entrées suivantes ?

   — Entrez le premier entier : 12
   — Entrez le deuxième entier : 2
   — Entrez l'opération (+, -, *, /) : *

   Conseil

   Consultez ce document sur le `switch` en Java. Référez-vous à la diapositive 23 du cours.

2. Que devez-vous modifier pour qu'il fonctionne ?

```
1   import java.util.Scanner;
2
3   public class Calculator {
4       public static void main(String[] args) {
5           Scanner sc = new Scanner(System.in);
6
7           System.out.print("Entrez le premier entier: ");
8           double num1 = sc.nextInt();
9
10          System.out.print("Entrez le deuxième entier: ");
11          double num2 = sc.nextInt();
12
13          System.out.print("Entrez l'opération (+, -, *, /): ");
14          char op = sc.next().charAt(0);
15
16          sc.close();
17
18          switch (op) {
19              case '+':
20                  System.out.println(num1 + num2);
21              case '-':
22                  System.out.println(num1 - num2);
23              case '*':
```

```
24                 System.out.println(num1 * num2);
25             case '/':
26                 System.out.println(num1 / num2);
27             default:
28                 throw new IllegalArgumentException("L'opération n'est
    pas valide");
29         }
30     }
31 }
```

Solution

```
Entrez le premier entier: 12
Entrez le deuxième entier: 2
Entrez l'opération (+, -, *, /): *
24.0
6.0
Exception in thread "main" java.lang.IllegalArgumentException:
L'opération n'est pas valide
at Calculator.main(Calculator.java:28)
```

Solution

Il faut simplement ajouter un `break;` après l'affichage du résultat d'une opération.

Question 14: (*5 minutes*) Qu'affiche le programme suivant, en supposant que l'utilisateur exécute le programme de la manière suivante dans le terminal ?

```
python3 question14.py 9
```

Conseil

Référez-vous à la diapositive 10 du cours.

```
1  import sys
2  if __name__ == '__main__':
3      balance = 20
4      try:
5          balance = balance - sys.argv[1]
6          solde_post = balance > 0
7          match (solde_post):
8              case True:
9                  print('Le solde de votre compte est positif.')
10             case False:
11                 print('Le solde de votre compte est négatif ou zéro.')
12                 raise ValueError('Le compte ne peut pas être débité')
13     except Exception as e:
14         print(f"Une erreur est survenue: {e}")
15     finally:
16         print(f"Valeur finale de balance: {balance}")
```

Solution

```
Une erreur est survenue
Valeur finale de balance: 20
```

Cette erreur est due à la soustraction d'une chaîne de caractères (`sys.argv[1]`) d'un nombre, ce qui n'est pas défini. Il faut donc utiliser `int()` pour convertir la chaîne de caractères en entier avant d'effectuer la soustraction.

Question 15: (*5 minutes*) Laquelle des affirmations suivantes est **fausse** concernant la différence entre une Error (erreur) et une Exception (exception) en Java ?

Conseil

Référez-vous à la diapositive 28 du cours.

(A) Les erreurs sont généralement causées par l'environnement dans lequel l'application s'exécute, tandis que les exceptions sont généralement causées par l'application elle-même.

(B) Les erreurs sont non vérifiées (*unchecked*) et n'ont pas besoin d'être déclarées dans la clause `throws` d'une méthode, tandis que les exceptions peuvent être soit vérifiées (*checked*), soit non vérifiées (*unchecked*).

(C) Les erreurs devraient généralement être interceptées et gérées par l'application afin d'assurer une récupération fluide, tandis que les exceptions peuvent être récupérables ou non.

(D) Les erreurs indiquent généralement des problèmes graves qu'une application raisonnable ne devrait pas essayer d'intercepter.

(E) `RuntimeException` sont des exceptions non vérifiées (*unchecked*).

Solution

(C) L'affirmation C est fausse car elle inverse l'approche habituelle. Les erreurs (telles que `OutOfMemoryError`, `StackOverflowError`) ne doivent généralement pas être détectées et gérées par les applications, car elles indiquent des problèmes graves qui ne peuvent généralement pas être résolus. Les exceptions, en revanche, sont conçues pour être détectées et gérées par l'application lorsque cela est approprié, car elles représentent souvent des conditions récupérables (telles que `ArithmeticException` ou `IllegalArgumentException`).